User Defined Functions (UDFs) in SQL Server

1. What is a User Defined Function?

A **User Defined Function (UDF)** is a **database object** in SQL Server that allows you to **encapsulate reusable logic** and return a value or a table.

It is similar to functions in programming languages like **C#, Java, or Python**.

Key Idea

A function:

- Accepts **input parameters**
- Performs **some logic**
- Returns **a result**

The result can be:

- A **single value**
- A **table**

---

2. Why Do We Use Functions?

Functions help to:

1. **Reuse logic**
2. **Reduce code duplication**
3. **Improve readability**
4. **Simplify complex queries**

Example:

Instead of calculating tax in multiple queries, create a function.

Tax = Salary * 0.1

Call the function wherever needed.

---

3. Types of User Defined Functions

SQL Server supports **three types of functions**:

1. **Scalar Function**
2. **Inline Table-Valued Function**
3. **Multi-Statement Table-Valued Function**

Let's understand each.

1️⃣ Scalar Function

Definition

A **Scalar Function** returns **a single value**.

Example:

Return

- Salary
- Total balance
- Age
- Tax
- Discount

Syntax

```
CREATE FUNCTION function_name
(
   @parameter datatype
)
RETURNS datatype
AS
BEGIN
```

```
    DECLARE @result datatype

    -- Logic

    RETURN @result

END
```

Example 1: Calculate Age

```
CREATE FUNCTION fn_GetAge
(
    @BirthDate DATE
)
RETURNS INT
AS
BEGIN

    DECLARE @Age INT

    SET @Age = DATEDIFF(YEAR, @BirthDate, GETDATE())

    RETURN @Age

END
```

How to Call Scalar Function

```
SELECT dbo.fn_GetAge('2000-05-10')
```

Output

25

Example 2: Calculate Interest

```
CREATE FUNCTION fn_CalculateInterest
(
    @Amount DECIMAL(10,2),
    @Rate DECIMAL(5,2)
)
RETURNS DECIMAL(10,2)
AS
BEGIN

    DECLARE @Interest DECIMAL(10,2)

    SET @Interest = @Amount * @Rate / 100

    RETURN @Interest

END
```

Calling Function

```
SELECT dbo.fn_CalculateInterest(10000, 5)
```

Output

Using Scalar Function in Query

Example: Employees table

```
SELECT
   EmployeeName,
   Salary,
   dbo.fn_CalculateInterest(Salary,10) AS Bonus
FROM Employees
```

2️⃣ Inline Table-Valued Function (ITVF)

Definition

An **Inline Table-Valued Function** returns a **table**.

It works similar to a **parameterized view**.

Important characteristics:

- Only **one SELECT statement**
- No BEGIN / END block
- Very **high performance**

Syntax

```
CREATE FUNCTION function_name
(
   @parameter datatype
)
RETURNS TABLE
AS
```

```
RETURN
(
    SELECT column1, column2
    FROM table
    WHERE condition
)
```

Example: Get Customer Accounts

Tables:

Customers

CustomerID

Name

Accounts

AccountID

CustomerID

Balance

Function:

```
CREATE FUNCTION fn_GetCustomerAccounts
(
    @CustomerID INT
)
RETURNS TABLE
AS
RETURN
(
```

```
    SELECT *
    FROM Accounts
    WHERE CustomerID = @CustomerID
)
```

Calling Inline Function

```
SELECT *
FROM dbo.fn_GetCustomerAccounts(101)
```

Output

AccountID	CustomerID	Balance
1	101	5000
2	101	8000

Using in JOIN

```
SELECT *
FROM dbo.fn_GetCustomerAccounts(101) a
JOIN Customers c
ON a.CustomerID = c.CustomerID
```

3️⃣ Multi-Statement Table-Valued Function (MSTVF)

Definition

A **Multi-Statement Table-Valued Function** returns a **table variable** and allows **multiple SQL statements**.

You can use:

- INSERT
- UPDATE

- DELETE
- Complex logic

Syntax

```
CREATE FUNCTION function_name
(
   @parameter datatype
)
RETURNS @table TABLE
(
   column1 datatype,
   column2 datatype
)
AS
BEGIN

   INSERT INTO @table
   SELECT columns
   FROM table

   RETURN

END
```

Example: Customer Account Summary

```
CREATE FUNCTION fn_CustomerBalanceSummary
(
    @CustomerID INT
)
RETURNS @Result TABLE
(
    AccountID INT,
    Balance DECIMAL(10,2),
    Status VARCHAR(20)
)
AS
BEGIN

    INSERT INTO @Result

    SELECT
        AccountID,
        Balance,
        CASE
            WHEN Balance > 5000 THEN 'High Balance'
            ELSE 'Low Balance'
        END
    FROM Accounts
    WHERE CustomerID = @CustomerID

    RETURN
```

END

Calling Function

SELECT *

FROM dbo.fn_CustomerBalanceSummary(101)

Output

AccountID	Balance	Status
1	6000	High Balance
2	2000	Low Balance

---

4. Difference Between Function Types

Feature	**Scalar**	**Inline TVF**	**Multi TVF**
Return type	Single value	Table	Table
Logic	Simple	One SELECT	Multiple statements
Performance	Slow	Fast	Moderate
Table variable	No	No	Yes

---

5. Function vs Stored Procedure

Feature	**Function**	**Stored Procedure**
Return value	Mandatory	Optional
Return type	Value / Table	Result set
Use in SELECT	Yes	No

Feature	Function	Stored Procedure
Use in WHERE	Yes	No
DML (Insert/Update/Delete)	Not allowed	Allowed

Example:

Valid:

SELECT dbo.fn_GetAge('2000-01-01')

Invalid:

SELECT * FROM EXEC GetCustomers

---

6. Important Rules for Functions

Functions **cannot perform certain operations**:

❌ Not Allowed

INSERT INTO permanent table

UPDATE table

DELETE table

EXEC stored procedure

TRY CATCH

Allowed:

SELECT

CASE

DECLARE variables

Table variables

---

7. Alter Function

Modify function:

```
ALTER FUNCTION fn_GetAge
(
    @BirthDate DATE
)
RETURNS INT
AS
BEGIN

    RETURN DATEDIFF(YEAR, @BirthDate, GETDATE())

END
```

---

8. Drop Function

Delete function:

```
DROP FUNCTION fn_GetAge
```

---

9. View Functions in Database

```
SELECT *
FROM sys.objects
WHERE type = 'FN'
```

---

10. Real Example (Banking Scenario)

Calculate **Total Balance of Customer**

```
CREATE FUNCTION fn_TotalBalance
(
    @CustomerID INT
)
RETURNS DECIMAL(10,2)
AS
BEGIN

    DECLARE @Total DECIMAL(10,2)

    SELECT @Total = SUM(Balance)
    FROM Accounts
    WHERE CustomerID = @CustomerID

    RETURN @Total

END
```

Use:

```
SELECT
    CustomerName,
    dbo.fn_TotalBalance(CustomerID) AS TotalBalance
FROM Customers
```

---

11. Best Practices

✓ Use **Inline Table Functions** whenever possible (best performance)

✓ Avoid **Scalar Functions inside large queries**

✓ Keep functions **simple**

✓ Use **schema prefix (dbo)**

Example:

dbo.fn_GetAge

---

12. Simple Visualization

```
User Defined Function
        |
        |
 ┌──────────────┬──────────────┬──────────────────┐
 Scalar Function  Inline Table Func   Multi Table Function
   (Single value)     (Table)              (Table)
```